Module 7 – Java – RDBMS & Database Programming with JDBC

1. Introduction to JDBC

- **Question:-1.** What is JDBC (Java Database Connectivity)?

Answer:- JDBC (Java Database Connectivity) is a Java API used to connect Java applications with databases. It allows Java programs to perform database operations such as:

Insert data , Update data , Delete data , Retrieve data

JDBC provides methods and interfaces to interact with databases like:

MySQL , Oracle , PostgreSQL , SQL Server

Using JDBC, Java applications can send SQL queries to the database and process the returned results.

- **Question:-2.** Importance of JDBC in Java Programming .

Answer:- 1. Database Connectivity
It helps Java applications communicate with databases.
2. Platform Independent
JDBC works on any platform where Java runs.
3. Supports SQL Queries
Developers can execute SQL commands directly from Java.
4. Data Manipulation
Allows CRUD operations:
Create , Read , Update , Delete
5. Secure Database Access

PreparedStatement prevents SQL Injection attacks.
6. Used in Enterprise Applications
Widely used in:

Banking systems , Web applications , ERP systems , Management software

➢ **Question:-3.** JDBC Architecture: Driver Manager, Driver, Connection, Statement, and ResultSet .

Answer:- JDBC architecture defines how Java applications communicate with databases.
Components of JDBC Architecture
1. Driver Manager
DriverManager manages JDBC drivers.
Responsibilities:
Loads database drivers
Establishes connection with database
Example:
DriverManager.getConnection(url, username, password);
2. JDBC Driver
A JDBC Driver is software that converts Java commands into database-specific commands.
Example:
MySQL JDBC Driver
Oracle JDBC Driver
3. Connection
Connection interface represents connection between Java application and database.
Example:
Connection con = DriverManager.getConnection(url,"root","1234");
Functions:
Open connection
Commit transactions
Close connection

4. Statement
Used to execute SQL queries.
Types:
Statement
PreparedStatement
CallableStatement
Example:
Statement st = con.createStatement();
5. ResultSet
Stores data returned from SELECT query.
Example:
ResultSet rs = st.executeQuery("select * from student");

2. JDBC Driver Types

- **Overview of JDBC Driver Types:**

➢ **Type:-1.** JDBC-ODBC Bridge Driver

Answer:- Converts JDBC calls into ODBC calls.

- Architecture
  Java Application → JDBC Driver → ODBC Driver → Database
- Advantages
  Easy to use
- Disadvantages
  Slow performance
  Requires ODBC installation
  Removed from Java 8

➢ **Type:-2.** Native-API Driver

Answer:- Uses native database libraries.

- Advantages
  Better performance than Type 1
- Disadvantages
  Database-specific

Native libraries required

- **Type:-3.** Network Protocol Driver.

  Answer:- Uses middleware server to communicate with database.

  - Advantages

    Database independent
  - Disadvantages

    Requires middleware server

- **Type:-4.** Thin Driver.

  Answer:- Directly communicates with database using native protocol.

  - Advantages

    Fastest

    Platform independent

    No middleware required
  - Disadvantages

    Database-specific driver needed
  - Example:

    com.mysql.cj.jdbc.Driver

- **Question:-2.** Comparison and Usage of Each Driver Type .

  Answer:-

Driver Type	Speed	Platform Independent	Middleware Required	Recommended
Type 1	Slow	Yes	Yes	No
Type 2	Medium	No	No	Rare
Type 3	Medium	Yes	Yes	Limited
Type 4	Fast	Yes	No	Yes

3. Steps for Creating JDBC Connections

- **Step-by-Step Process to Establish a JDBC Connection:**

➢ **Question:-1.** Import the JDBC packages.

Answer:- import java.sql.*;

➢ **Question:-2.** Register the JDBC driver.

Answer:- Class.forName("com.mysql.cj.jdbc.Driver");

➢ **Question:-3.** Open a connection to the database .

Answer:-

```
Connection con = DriverManager.getConnection
("jdbc:mysql://localhost:3306/college","root","");
```

➢ **Question:-4.** Create a statement.

Answer:- Statement st = con.createStatement();

➢ **Question:-5.** Execute SQL queries.

Answer:-ResultSet rs = st.executeQuery("select * from student");

➢ **Question:-6.** Process the resultset.

Answer:-

```
while(rs.next()){
   System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

Question:-7. Close the connection.

Answer:- con.close();

4. Types of JDBC Statements

• Overview of JDBC Statements:

➢ **Question:-1.** Statement: Executes simple SQL queries without parameters.

Answer:-

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("select * from student");
```

➢ **Question:-2.** PreparedStatement: Precompiled SQL statements for queries with parameters.

Answer:-

- Advantages
  Faster execution
  Prevents SQL injection
- Example

```
PreparedStatement ps =
        con.prepareStatement("insert into student
                                        values(?,?)");

ps.setInt(1,101);
ps.setString(2,"Parth");
```

➢ **Question:-3.** CallableStatement: Used to call stored procedures.

Answer:-

CallableStatement cs =con.prepareCall("{call getStudent()}");

- **Question:-4.** Differences between Statement, PreparedStatement, and CallableStatement

  Answer:-

Feature	Statement	PreparedStatement	CallableStatement
Query Type	Simple	Parameterized	Stored Procedure
Compilation	Every time	Precompiled	Precompiled
Performance	Slow	Faster	Fast
Security	Less secure	More secure	Secure
Parameters	No	Yes	Yes

5. JDBC CRUD Operations (Insert, Update, Select, Delete)

- **Question:-1.** Insert: Adding a new record to the database.

  Answer:- insert into student values(101,'Parth',85);

  PreparedStatement ps = con.prepareStatement("insert into student values(?,?,?)");

```
ps.setInt(1,101);
ps.setString(2,"Parth");
ps.setInt(3,85);

ps.executeUpdate();
```

- **Question:-2.** Update: Modifying existing records.

  Answer:- update student set marks=90 where id=101;

```
PreparedStatement ps =
con.prepareStatement(
"update student set marks=? where id=?");

ps.setInt(1,90);
ps.setInt(2,101);

ps.executeUpdate();
```

- **Question:-3.** Select: Retrieving recordsfrom the database.

  Answer:- select * from student where marks>80;

```
Statement st = con.createStatement();

ResultSet rs =
st.executeQuery("select * from student");

while(rs.next())
{
   System.out.println(rs.getInt(1));
}
```

- **Question:-4.** Delete: Removing records from the database.

  Answer:- delete from student where id=101;

```
PreparedStatement ps =
con.prepareStatement(
"delete from student where id=?");

ps.setInt(1,101);

ps.executeUpdate();
```

6. ResultSet Interface

- **Question:-1.** What is ResultSet in JDBC?

  Answer:- ResultSet is an object that stores records returned from SELECT query.

  ResultSet rs = st.executeQuery("select * from student");

- **Question:-2.** Navigating through ResultSet (first, last, next, previous)
- **Answer:**- rs.next(); , rs.first(); , rs.last(); , rs.previous();
- **Question:-3.** Working with ResultSet to retrieve data from SQL queries .

  Answer:-

```
while(rs.next())
   {
      int id = rs.getInt("id");
      String name = rs.getString("name");

      System.out.println(id+" "+name);    }
```

7. Database Metadata

- **Question:-1.** What is DatabaseMetaData?

  Answer:-

  DatabaseMetaData provides information about database.

  Example:
  Database name ,Version , Tables , Driver details

- **Question:-2.** Importance of Database Metadata in JDBC .

Answer:-

- Analyze database structure
- Get table information
- Get supported SQL features

- **Question:-3.** Methods provided by DatabaseMetaData (getDatabaseProductName, getTables, etc.)

  Answer:-

```
DatabaseMetaData dm = con.getMetaData();

System.out.println(dm.getDatabaseProductName());
ResultSet rs =
```

dm.getTables(null,null,null,null);

8. ResultSet Metadata

- **Question:-1.** What is ResultSetMetaData?

  Answer:- Provides information about ResultSet structure.

  Example:

  Column count
  Column names
  Column types

- **Question:-2.** Importance of ResultSet Metadata in analyzing the structure of query results .

  Answer:- Dynamic table generation
  Generic applications
  Reporting tools

- **Question:-3.** Methods in ResultSetMetaData (getColumnCount, getColumnName,
- getColumnType).

  Answer:- ResultSetMetaData rm = rs.getMetaData();

  System.out.println(rm.getColumnCount());
  System.out.println(rm.getColumnName(1));
  System.out.println(rm.getColumnType(1));

9. Practical SQL Query Examples

- **Write SQL queries for:**

➢ **Question:-1.** Inserting a record into a table .

Answer:-

```
insert into student(id,name,city,marks)
values(101,'Parth','Ahmedabad',85);
import java.sql.*;

public class InsertDemo
{
   public static void main(String[] args)
   {
      try
      {
         // Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // Create Connection
         Connection con =
         DriverManager.getConnection(
         "jdbc:mysql://localhost:3306/college",
         "root",
         "1234");

         // Create PreparedStatement
         PreparedStatement ps =
         con.prepareStatement(
         "insert into student values(?,?,?,?)");

         // Set Values
         ps.setInt(1,101);
```

```
            ps.setString(2,"Parth");
            ps.setString(3,"Ahmedabad");
            ps.setInt(4,85);

            // Execute Query
            int i = ps.executeUpdate();

            if(i>0)
            {
               System.out.println("Record Inserted");
            }

            // Close Connection
            con.close();
         }
         catch(Exception e)
         {
            System.out.println(e);
         }
      }
   }
```

- **Question:-2.** Updating specific fields of a record .

Answer:-

```
update student
set marks=90
where id=101;
import java.sql.*;

public class UpdateDemo
{
   public static void main(String[] args)
   {
```

```
        try
        {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con =
            DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college",
            "root",
            "1234");

            PreparedStatement ps =
            con.prepareStatement(
            "update student set marks=? where id=?");

            ps.setInt(1,90);
            ps.setInt(2,101);

            int i = ps.executeUpdate();

            if(i>0)
            {
                System.out.println("Record Updated");
            }

            con.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

- **Question:-3.** Selecting records based on certain conditions .

Answer:- select * from student
where marks > 80;

```
import java.sql.*;

public class SelectDemo
{
   public static void main(String[] args)
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");

         Connection con =
         DriverManager.getConnection(
         "jdbc:mysql://localhost:3306/college",
         "root",
         "1234");

         Statement st =
         con.createStatement();

         ResultSet rs =
         st.executeQuery(
         "select * from student where marks>80");

         while(rs.next())
         {
            System.out.println(
            rs.getInt("id")+" "+
```

```
            rs.getString("name")+" "+
            rs.getString("city")+" "+
            rs.getInt("marks"));
        }

        con.close();
      }
      catch(Exception e)
      {
        System.out.println(e);
      }
    }
}
```

- **Question:-4.** Deleting specific records .

Answer:- delete from student
where id=101;

```
import java.sql.*;

public class DeleteDemo
{
   public static void main(String[] args)
   {
      try
      {
         Class.forName("com.mysql.cj.jdbc.Driver");

         Connection con =
         DriverManager.getConnection(
         "jdbc:mysql://localhost:3306/college",
         "root",
         "1234");
```

```
            PreparedStatement ps =
            con.prepareStatement(
            "delete from student where id=?");

            ps.setInt(1,101);

            int i = ps.executeUpdate();

            if(i>0)
            {
               System.out.println("Record Deleted");
            }

            con.close();
         }
         catch(Exception e)
         {
            System.out.println(e);
         }
      }
}
```

- **Question:-5.** Implement these queries in Java using JDBC .

Answer:- JDBC allows Java applications to communicate with databases efficiently. Using PreparedStatement, developers can safely and easily perform CRUD operations like:

INSERT
UPDATE
SELECT

DELETE

It provides:

Better performance
High security
Cleaner code
Easy handling of dynamic data

10. Practical Example 1: Swing GUI for CRUD Operations

- **Question:-1.** Introduction to Java Swing for GUI development.

  Answer:- Swing is Java GUI toolkit used to create desktop applications.

  Components:

  JFrame ,JButton , JTextField , JTable

- **Question:-2.** How to integrate Swing components with JDBC for CRUD operations.

  Answer:- Swing forms can perform CRUD operations using JDBC.
  Example:
  User enters data in text field
  JDBC inserts data into database

11. Practical Example 2: Callable Statement with IN and OUT Parameters

- **Question:-1.** What is a CallableStatement?

  Answer:- CallableStatement is used to call stored procedures from Java.

- **Question:-2.** How to call stored procedures using CallableStatement in JDBC .

  Answer:-

```
create procedure getStudent()
begin
select * from student;
end
```

- **Question:-3.** Working with IN and OUT parameters in stored procedures .

  Answer:-

```
create procedure squareNum(
in num int,
out result int)

begin
set result = num*num;
end
```